Uber Fare Prediction

Estimate a historical ride fare from pickup and drop-off location, passenger count, and pickup time.

Objective

Pre-process the fare dataset, identify and report statistical outliers, examine feature correlations, train Linear Regression and Random Forest Regression, then compare test-set R2, RMSE, and MAE.

Dataset at a glance

Item	Description
Source	Uber Fares Dataset, Kaggle (Yasser H).
Raw records	200,000 rides; one row had missing drop-off coordinates.
Cleaned records	195,108 rides after missing/duplicate checks and validity filters.
Target	fare_amount, measured in USD.
Predictors	13 numeric route, passenger, and pickup-time features.

Dataset source: kaggle.com/datasets/yasserh/uber-fares-dataset

Evaluation uses a reproducible random 80/20 split. The scores describe this cleaned historical sample, not a live fare quote.

Pre-processing and feature design

Invalid records were removed first; plausible high fares flagged by the IQR rule were retained for modeling.

Cleaning and outlier analysis

The index-like column and ride key were excluded from the predictors. Missing coordinates and duplicate records were checked. Rows with non-positive or implausibly high fares, geographically implausible NYC coordinates, or invalid passenger counts were filtered. This left 195,108 rows for analysis.

Fare outlier check	Result
Q1 / Q3	USD 6.00 / USD 12.50
IQR fences	USD -3.75 to USD 22.25
Flagged fares	16,713 rides (8.57% of cleaned rows)
Treatment	Reported and retained after validity checks

The IQR rule marks unusually high fares relative to the sample; it does not prove that a ride record is erroneous. Retaining plausible long trips avoids removing valid examples solely because their fares are large.

Location and time features

The Haversine formula converts pickup and drop-off coordinates into great-circle distance in kilometers. Pickup hour, weekday, and month are encoded cyclically with sine and cosine so that adjacent periods remain close in feature space. The four raw coordinates, passenger count, distance, six cyclic time features, and pickup year form the 13 model features.

```
lat1 = np.radians(data["pickup_latitude"])
lat2 = np.radians(data["dropoff_latitude"])
lon1 = np.radians(data["pickup_longitude"])
lon2 = np.radians(data["dropoff_longitude"])
a = np.sin((lat2-lat1)/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin((lon2-lon1)/2)**2
data["distance_km"] = 2 * 6371.0 * np.arcsin(np.sqrt(np.clip(a, 0, 1)))
```

Test-set performance

The same 39,022 held-out rides were used to evaluate both regression models.

Correlation check

Feature	Pearson correlation with fare
Distance (km)	+0.823
Pickup longitude	+0.361
Drop-off longitude	+0.265
Pickup year	+0.120
Pickup latitude	-0.186
Other listed time/passenger features	-0.021 to +0.024

Distance has the strongest linear association with fare in this feature check. Correlation is descriptive and does not establish causation; nonlinear effects and location patterns can still matter.

Training and scores

The training set contains 156,086 rides and the test set 39,022 rides. Linear Regression is placed after StandardScaler in a pipeline. The Random Forest uses 100 trees, maximum depth 18, minimum leaf size 2, and random seed 42.

```
linear = make_pipeline(StandardScaler(), LinearRegression())
forest = RandomForestRegressor(
    n_estimators=100, max_depth=18, min_samples_leaf=2,
    n_jobs=-1, random_state=42
)
# Fit both models on the same training rows; score on held-out rows.
```

Model	R2 (higher is better)	RMSE (USD, lower)	MAE (USD, lower)
Linear Regression	0.6904	5.5376	2.3984
Random Forest	0.8336	4.0603	1.8388

Random Forest performed better on this test split: it had the higher R2 and lower RMSE and MAE. Its RMSE was about USD 4.06 per ride and its MAE about USD 1.84.

Conclusion

The results support Random Forest for this comparison because it captures nonlinear fare patterns more effectively than the linear baseline. Performance may change with another split, geography, time period, or deployment data. A time-ordered validation would be needed to estimate performance on future rides.